



Automated generation of mechanistic models for chemical process digital twins using reinforcement learning part II: Compartmentalization and learning-based recalibration

Jan-Frederic Laub^{a,b}, Jiyizhe Zhang^{a,c,*}, Mathis Heyer^{a,b},
Alexei A. Lapkin^{a,c,d,*}

^a Department of Chemical Engineering and Biotechnology, University of Cambridge, CB3 0AS, United Kingdom

^b Process Systems Engineering (AVT.SVT), RWTH Aachen University, 52074 Aachen, Germany

^c Innovation Centre in Digital Molecular Technologies, Yusuf Hamied Department of Chemistry, University of Cambridge, Lensfield Road, Cambridge, CB2 1EW, United Kingdom

^d Cambridge Centre for Advanced Research and Education in Singapore Ltd., 1 Create Way, CREATE Tower #05-05, Singapore 138602, Singapore

ARTICLE INFO

Keywords:

Automated modeling
Reinforcement learning
Mechanistic models
Transfer learning
Digital twins

ABSTRACT

Developing predictive models is central to building digital twins for chemical processes, which have a variety of applications in their development and operation. Mechanistic models are highly interpretable and have a larger domain of validity compared to data-driven models, but require significant time and expert knowledge to construct. In this contribution, a workflow for automated mechanistic model generation is extended to handle systems comprised of interdependent, spatially distributed phenomena. The search for accurate models is performed by hierarchically connected reinforcement learning agents. Different ways to incorporate human expertise in model generation are explored, and an ontology is introduced to manage expert and modeling knowledge. The extended workflow is shown to reliably find accurate models of chemical systems, exemplified on a phase transfer catalysis reaction and a Taylor-Couette reactor. For the latter, its non-ideal flow patterns were predicted within a deviation of 5 %, and automatically generated compartmentalization results were found to have comparable physical interpretations to bespoke models from literature. Additionally, the reinforcement learning agents were able to accurately recalibrate models up to twice as fast when drawing upon pre-training under a different operation condition. By generalizing all parts of the automated modeling procedures, we enable the efficient (re-)use of knowledge previously confined to the human modeler. We envision that in the future, the role of experts can be shifted from actively constructing each model iteration to curating knowledge and working collaboratively with autonomous agents.

1. Introduction

The chemical industry is undergoing a significant transformation towards sustainable and resource-efficient manufacturing. To achieve this goal, many novel reactor and process designs have been developed, often involving elaborate technical solutions, such as multi-phenomena process intensification, miniaturized flow reactors and separators, or the use of non-thermal molecular activation methods like photo- and electrochemistry (Stankiewicz and Moulijn, 2000; Tian et al., 2018). Vendors of these advanced chemical process units face the challenge of frequently demonstrating capabilities under various operating conditions and for a variety of chemical systems, which is both laborious and

expensive.

Digital twins have emerged as valuable tools to address this challenge (Mane et al., 2024). A digital twin is composed of a model and associated information that represent the novel unit operation in a digital environment at different stages of the physical system's life cycle (Grieves, 2023; Pal et al., 2025). Having a reliable digital twin enables, for example, the simulation and rapid evaluation of new designs or process conditions, thereby reducing the reliance on costly real-world experimentation in process development. It also empowers device manufacturers to digitally showcase the advantages of their innovations.

At the core of a digital twin is a predictive model that captures the behavior of the target system with sufficient accuracy. Mechanistic

* Corresponding authors.

E-mail addresses: jz596@cam.ac.uk (J. Zhang), aal35@cam.ac.uk (A.A. Lapkin).

<https://doi.org/10.1016/j.compchemeng.2025.109384>

Received 27 March 2025; Received in revised form 16 August 2025; Accepted 30 August 2025

Available online 1 September 2025

0098-1354/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

models are often preferred for their interpretability and ability to extrapolate beyond the scope of the observed data. However, developing mechanistic models demands extensive expert knowledge and time (Gargalo et al., 2024) hindering their industrial adoption in the fast-paced environment of chemical research and development. For specialized applications, e.g., determining kinetic relationships, there exist well-established evolutionary approaches (for example De Carvalho Servia et al., 2024) that, however, lack the generality to describe arbitrary unit operation designs. There also exist more generalized modeling frameworks (for example Demirel et al., 2019; Freund and Sundmacher, 2008). However, these are often too sophisticated for practical use, computationally intensive, or not well-suited for adaptive recalibration of the model under changing conditions, which is a critical feature in digital twin applications.

To address the central challenges above, this two-part work presents an automated workflow for generating mechanistic models using reinforcement learning (RL). Its novelty is in combining the different hierarchical levels of mechanistic model generation without the need for excessive superstructure optimization, while leveraging the flexibility of reinforcement learning for the integration of expert knowledge and model recalibration. This second part specifically focuses on extending the workflow to complex unit operations comprised of multiple interdependent and spatially distributed phenomena, standardizing expert modeling knowledge in an ontology, and reusing RL agents to speed up model generation under changing operating conditions.

In the first part of this serial work, Heyer et al. (2025) have designed a generalized and modular workflow for automated model generation and tested it on a flow reactor. The workflow proposes several consecutive modules. First, an input module is provided with experimental data and domain knowledge. The information is then used by an automatic compartmentalization module to identify a candidate topology that describes the system's structure, e.g., a subdivision of a reactor into smaller balancing volumes (Haag et al., 2018; Jourdan et al., 2019). This compartmentalization module was not specified further in Part I and is the main subject of this contribution. Next, the equation generator decides on the differential-algebraic description of the topology's subunits by sequentially determining which physical quantities can be neglected, substituted by constitutive equations or known values, or should be estimated from data. The finished candidate model must pass a test module before it is solved, and model parameters are estimated if necessary. Finally, the accuracy of the candidate model is evaluated with respect to experimental data, and the model's error is propagated back to guide further iterations of topology and equation generation.

In this second part, we focus on extending the workflow's compartmentalization procedures to model novel designs of complex unit operations. We leverage reinforcement learning to generate accurate model topologies and facilitate their recalibration under changing operating conditions. Different formats for incorporating expert knowledge, such as graph grammar and reward shaping, are explored to guide the RL agents. We then design an ontology to save and standardize this knowledge. The performance of the workflow is tested on two case studies: one in silico case study focusing on phase transfer catalysis, and one experimental case study with a novel design of a continuous flow reactor provided by a vendor. The workflow is also evaluated for its ability to recalibrate models under changing operating conditions via transfer learning.

2. Methodology

The methods developed in this work aim to enable flexible and holistic automated modeling of novel designs of unit operations. In Section 2.1, we define general building blocks from which we can generate compartments and interfaces of the to-be-generated model topologies. Then, Section 2.2.1 translates these topologies to states in a Markov Decision Process by describing them as colored digraphs. In Section 2.2.2, we introduce graph grammar as a standardizable way to shape

these graphs towards accurate system descriptions. The search for accurate topologies is facilitated by shaping the reward function of the decision process with the aid of system knowledge, as described in Section 2.2.3. All methods developed in this work are fully integrated into the framework presented in Part 1 (Heyer et al., 2025), and the hierarchical relationship between the two RL agents is detailed in Section 2.3.1. Finally, Section 2.3.2 conceptualizes how artifacts of the automated model generation can be related to system knowledge through an ontology. Fig. 1 shows an overview of the overall workflow and the extensions addressed in this work.

2.1. Model topology and building blocks

Complex chemical systems are commonly modeled as arrangements of simpler compartments that are connected by interfaces to each other and the environment (Gilles et al., 1986). In the following, we refer to such a network of compartments and interfaces as a model topology M . The sets of all compartments and interfaces constituting a topology are denoted by C and I . To compute predictions from a given model, its topology must first be translated into a set of balance and coupling equations. We perform this translation based on information contained in preconfigured building blocks.

Building blocks are general classes of topological elements from which the specific compartments and interfaces of a model inherit their properties. For example, a compartment could inherit the properties of a CSTR building block while an interface might extend a two-film-connection building block. The resulting equations represent the basic mechanistic scaffold of a model. This scaffold can then be specified towards the occurring phenomena, like reactions or mass transfer, by the insertion of constitutive relationships and experimentally determined parameters.

Inspired by the description of elementary process functions by Freund and Sundmacher (2008), we define a general building block, see Fig. 2, as a balance volume and a set of fluxes that can represent transmissions of mass, energy, or momentum.

The set of all available building blocks for a given application is $B = B_C \cup B_I$. We define a function $\phi: C \cup I \rightarrow B$ to assign building blocks to the compartments and interfaces of a topology. A model topology can thus be represented as the tuple $M = (C, I, B, \phi)$.

To generate the equation scaffold of a model, every compartment of the model topology concatenates the equations inherited from its building block to the overall set of equations, e.g., for mass (b_m) and energy (b_e) balances:

$$b_{m,c}(F_c, J_c) = 0 \quad \forall c \in C, \quad (1)$$

$$b_{e,c}(F_c, J_c) = 0 \quad \forall c \in C. \quad (2)$$

Similarly, the interfaces of the topology generate coupling constraints via their respective balances that relate the fluxes of the connected compartments ($c_{s,i}$ and $c_{t,i}$ for interface i) to each other:

$$s_i(F_{c_{s,i}}, F_{c_{t,i}}, J_{c_{s,i}}, J_{c_{t,i}}) = 0 \quad \forall i \in I. \quad (3)$$

A model topology M becomes a fully specified model $\hat{M}$ after the equation generator of Part 1 has added appropriate parameter values and constitutive equations $c(F, J)$, which is detailed in Section 2.3.1.

In our process of describing a physical system with a model topology, we follow a similar approach to Khan and Lapki, (2020), where we center our perspective on the system's streams. We can fully describe these streams by their mass composition, flow rate, and thermodynamic properties, such as pressure and temperature. We initialize a to-be-generated model topology with a compartment representing the environment (ENV), which is everything not explicitly resolved by the system's model. Initially, the physical streams of the system are represented as (usually convective) interfaces, starting from and ending in the environment compartment. Over the course of model generation,

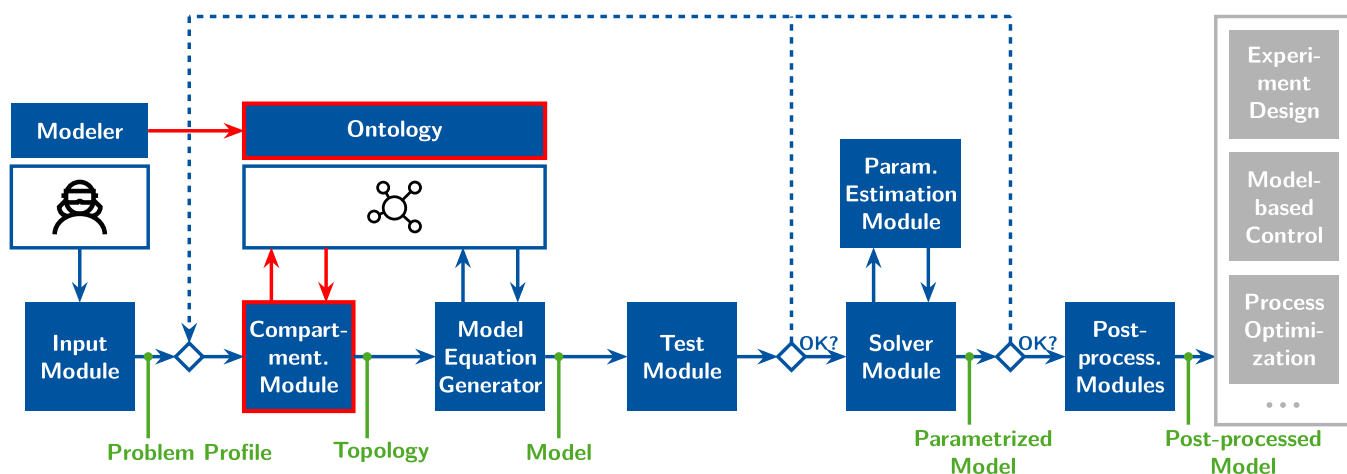


Fig. 1. An illustration of the automated model generation workflow. In blue: conceptual workflow modules from Part 1, highlighted in red: modules and connections developed in this work, in green: workflow outputs, in grey: possible applications.

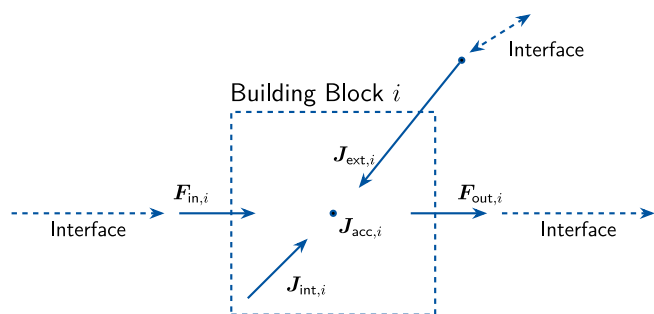


Fig. 2. General example of a building block for a compartment. It hosts the incoming and outgoing fluxes F_{in} and F_{out} , the internal fluxes J_{int} (e.g., reactions), and the external fluxes from other compartments J_{ext} (e.g., mass transfer). The high-level distinction between F and J_{ext} is made here to differentiate between convection- and diffusion-dominated fluxes which simplifies the implementation. Additionally, we generalize the storage as the accumulation flux J_{acc} . A general building block for an interface, i.e., a topological unit that connects two compartments, only consists of an incoming and outgoing flux.

compartments and interfaces are iteratively added to the physical streams' virtual representations until their predicted properties match experimental measurements; see Fig. 3. The experimentally observed stream properties are collected in D_{measured} while the corresponding model predictions D are a function of the fluxes entering the environment compartment, i.e., $D = D(F_{\text{in,ENV}})$.

2.2. Formulating model generation as a Markov decision process

The task of topology generation is formulated as a Markov Decision Process (MDP), emulating the sequential decision-making of expert-driven compartmental modeling. This makes the automated modeling more transparent and explainable to human observers. In comparison to established methods based on superstructure optimization and mixed-integer nonlinear programming (MINLP), the MDP does not require an *a priori* grid structure or fixed model sizes, thereby offering increased flexibility and reduced computational effort, as not every possible combinatorial model composition is implicitly considered. The flexibility of this formulation becomes especially useful when combining modeling decisions of different hierarchies, see [Section 2.3.1](#).

At its core, the topology generation problem is cast as an MDP in which the agent sequentially interacts with the problem environment. A state $s \in S$ represents the current model topology, encoded as a colored

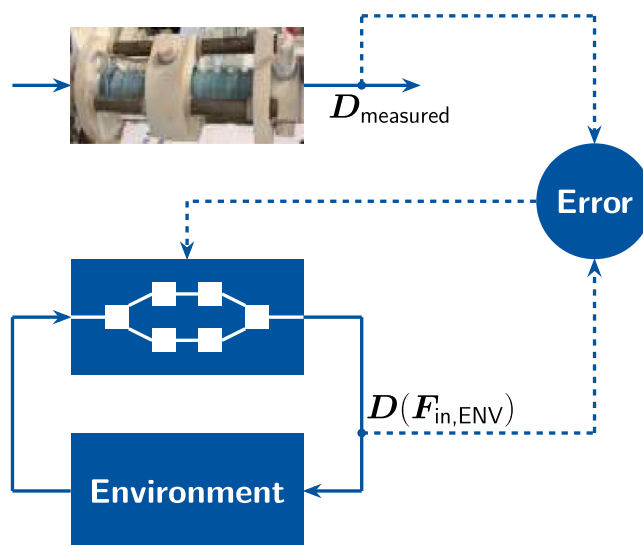


Fig. 3. The properties of the physical system's outgoing streams are measured and compared to the predictions of the model. The predictions are taken from the fluxes of the interface entering the environment compartment. The prediction error guides further refinement of the compartmental model.

digraph. From each state, the agent selects an action $a \in A(s)$, typically a graph manipulation such as inserting, connecting, or rearranging compartments and interfaces, which transitions the environment to a new state s' . After each action, the agent receives a reward $R(s, a)$ measuring how accurately the resulting topology represents the physical system. The tuple (S, A, R) therefore fully defines the MDP.

A policy $\pi(a|s)$ prescribes the agent’s actions in each state, while the action-value function $Q_\pi(s, a)$ returns the expected cumulative reward from taking action a in state s and thereafter following π . By exploring and exploiting the search space, the agent populates a Q-table that approximates Q_π and acts as a global memory, enabling knowledge transfer between different model generation campaigns (e.g., recalibration, [Section 3.2.4](#)). This experience reuse distinguishes the method from evolutionary approaches like genetic algorithms, which typically do not retain learning beyond warm-starting.

The generated topologies are translated into mathematical equations (Section 2.1) and refined using the equation generator from Part 1. These equations are then embedded in a parameter estimation problem, with the resulting predictive accuracy contributing to the reward signal. This coupling of graph-based topology search with quantitative

performance evaluation allows reinforcement learning to systematically navigate the vast design space of mechanistic models.

While reinforcement learning for graph construction has been explored in other domains (e.g., molecule generation (Mazuz et al., 2023) or infrastructure networks (Darvari et al., 2021)), our work adapts this paradigm to the domain of mechanistic chemical reactor modeling. This involves coupling the discrete, structural search space of compartmental topologies with the continuous, parameter-driven evaluation of their predictive accuracy. In this way, the approach leverages RL's ability to explore large combinatorial spaces while systematically incorporating mechanistic constraints and experimental data into the reward structure. The novelty lies not in advancing RL theory itself, but in formalizing how these established methods can be deployed for system identification in chemical engineering, including the integration of hierarchical decision-making and parameter estimation workflows.

In the following, we detail the formalization of states, actions, and rewards in our topology generation MDP, explain how graph production rules are defined and applied, and describe how parameter estimation and model evaluation are embedded into the reinforcement learning loop.

2.2.1. States: candidate models with topologies represented as colored digraphs

The model topologies, consisting of compartments and interfaces that inherit their properties from globally defined building blocks, are abstracted as colored directional graphs (digraphs). Graphs are an established tool to model, analyze, and visualize systems in chemical engineering, and are a proven way to make model topologies machine-readable (Mah, 1983).

A colored digraph is a 5-tuple $G = (V, E, \varphi, C_V, C_E)$, where V is a finite set of vertices, $E \subseteq V \times V$ is a finite set of directed edges, C_V is a finite set of vertex colors, C_E is a finite set of edge colors, and $\varphi : V \cup E \rightarrow C$ is a color function, where $\varphi_V : V \rightarrow C_V$ assigns a color from C_V to each vertex, and $\varphi_E : E \rightarrow C_E$ assigns a color from C_E to each edge. We identify the topological elements introduced in Section 2.1 with the graph elements, i.e., the model compartments with the vertices $V \equiv C$ and the model interfaces with the edges $E \equiv I$. The pre-defined classes of building blocks are represented as colors, i.e., $C_V \equiv B_C$ and $C_E \equiv B_I$. Correspondingly, the function of assigning these classes is equivalent to the color assignment function $\varphi \equiv \phi$. By abstracting the model topology as a graph, the task of topology generation is reformulated into a graph generation problem. Fig. 4 shows a conversion from an exemplary system (in this case a heterogeneous reactor) to a model topology represented as a colored digraph.

Independent of how graph generation is performed specifically, there must always be a way to identify a given graph, as the topology generation agent needs to recognize in which state the environment is and which actions are allowed to be applied to that state next. Therefore, we require:

- (1) an easy-to-store, canonical reference to specific graphs, and
- (2) a canonical labeling of the graph elements such that manipulations can be made to specific location within a graph.

If we simply store a graph by the enumeration of its elements, several descriptions of the same structure are possible, thereby requiring complicated and inelegant comparison algorithms. Instead, we turn to algorithms developed in the field of graph canonization. First, we adopt an equality condition for graphs: two graphs are equal if they belong to the same isomorphism class. Graph canonization algorithms can find a canonical representation of the isomorphism class of a given graph in the form of a string. For this task, we choose *scott* (Bloyet et al., 2020) because it can handle both vertex- and edge-colored graphs out-of-the-box. In addition to *scott*, we use the package *bliss* (Junttila and Kaski, 2007) to obtain a canonical relabeling of the colored edges and vertices. The computational complexity of canonizing graphs of

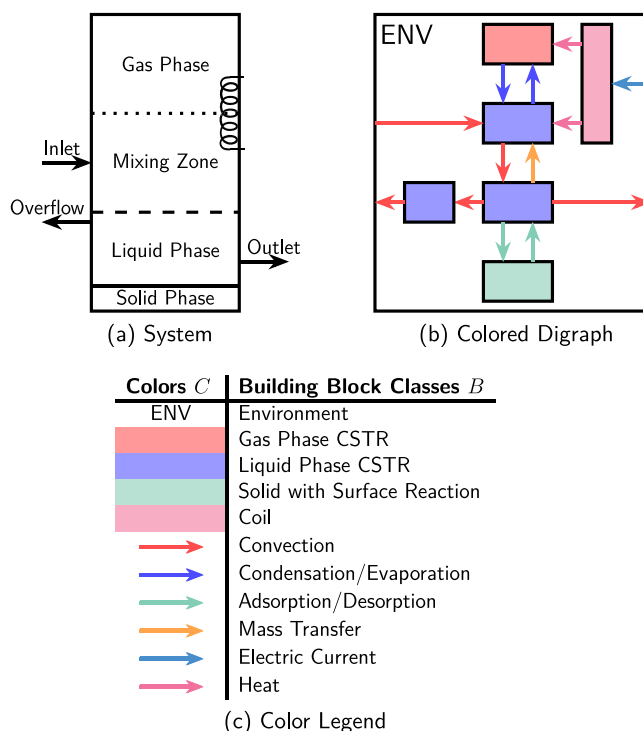


Fig. 4. An illustration of the translation of an example system to a model topology represented by a colored digraph.

arbitrary sizes is still an open question, although there are some indications that it belongs to the class of polynomial problems (Babai, 2016). For the relatively small graphs that we expect to encounter in this work, we can safely assume that the canonization algorithms require a negligible fraction of the acceptable computational effort.

Combined with the constitutive equations and parameter values chosen by the equation generator (see Section 2.3.1), a model topology becomes a candidate model $\hat{M}$, and forms a state in the MDP.

2.2.2. Actions: graph manipulation based on graph grammar

The actions of the topology generation MDP are the different ways the model topologies can be manipulated through the addition, deletion, and connection of compartments and interfaces. In similar works on graph generation (Darvari et al., 2021; Stops et al., 2023), these manipulation rules are kept relatively simple and can therefore be implemented in a trivial manner. A core idea of this work is to formalize the graph manipulation rules such that:

- (1) more complicated rules can be executed in standardized ways, and
- (2) different sets of manipulation rules can be stored and assessed depending on the type of system that should be modeled.

To formalize these construction rules, we borrow the concept of productions from the field of graph grammar. A production p is represented by a tuple of graphs (L, R) . Subjecting a graph G to a production p means that if L is a subgraph of G , then the corresponding elements of L in G can be removed and replaced by the graph elements from R . In accordance with common notation in this field, we write $p : L ::= R$. For our purposes, a set of graph productions P forms a graph grammar. For a more rigorous introduction to graph grammar, see Ehrig et al. (1992). Given an initial graph G_0 , the set of all graphs that are attainable by the successive application of productions in P is called a graph language. Therefore, the graph language corresponds to the attainable space of model graphs, i.e., the search space of possible model topologies. By shaping the graph language via its grammar, we can influence

which kinds of model graphs are included in and excluded from that search space. We denote the set of productions that can be applied to the topology of a state s as $P(s)$ which forms the set of possible actions $A(s)$ of the MDP.

In generating models for a unit operation, we typically aim to obtain structures that have a fixed number of incoming and outgoing interfaces corresponding to the known streams. With an appropriate graph grammar, we can discriminate the search space towards these types of topologies. The graph grammar described in Fig. 5 enables us to restrict the search space to models that only possess the known streams and always ensure that an incoming stream from the environment also leaves the system. That way, we can always identify the experimentally observed system states to later fit the correct model parameters.

This grammar alone is very restrictive and would not enable modeling common phenomena, for example, the mixing of two streams. However, because we formulated graph manipulation rules in this systematic way, we can easily extend the grammar with more productions tailored to specific case studies. Using graph grammar further modularizes the topology generation process and enables future analysis of adequate graph grammars for different physical systems.

2.2.3. Rewards: accuracy-based and knowledge-shaped

The topology generation MDP is finalized by defining the reward that is distributed to the agent after the model generation steps. By consciously shaping that reward, we can condition the agent to efficiently explore large and sparse search spaces by guiding it with expert knowledge. We first recognize that the distributed reward is entirely dependent on the current state of the MDP and is not influenced by the specific action that led to this state: $R(s, a) := R(s)$. Then, we identify the state with the generated candidate model as explained in Section 2.2.1, i.e., $R(s) := R(\hat{M})$. The reward function is split into two parts, one that depends directly on the topology (R_1), and one that is determined after evaluating the model against the experimental data of the physical twin (R_2). Together, they form the total reward distributed to the state-action pairs: $R(\hat{M}) = R_1(\hat{M}) + R_2(ERR(\hat{M}))$.

The error $ERR(\hat{M})$ of a given model $\hat{M} = (C, I, B, \phi, c)$ is determined

by fitting the model parameters to experimental data by minimizing the sum of square errors of the predicted minus the measured stream states:

$$ERR(\hat{M}) = \min_{F, J} \|D(F_{in, ENV}) - D_{measured}\|_W^2 \quad (4)$$

$$\text{s.t. } b_c(F_c, J_c) = 0 \quad \forall c \in C, \quad (5)$$

$$s_i(F_{c_{s_i}}, F_{c_{t_i}}, J_{c_{s_i}}, J_{c_{t_i}}) = 0 \quad \forall i \in I, \quad (6)$$

$$c(F, J) = 0. \quad (7)$$

The constitutive relationships c , which also include setting known parameters, are obtained from executing the equation generator described in Part 1. We further discuss its integration with the topology generation in Section 2.3.1. In the objective, $\|\cdot\|_W$ represents the weighted Euclidean norm with the diagonal weight matrix W that normalizes the different measurement quantities by the average of their measurement values. For better scaling, the determined model error is transformed into a reward between 0 and 1 with the following formula from Part 1:

$$R_2(ERR) = \frac{1}{1 + ERR}. \quad (8)$$

The first part of the reward function R_1 is constructed by incorporating formalized system knowledge. That way, the agent can learn to recognize parts of the search space where expert knowledge leads to reasonable expectations of accurate models. Naturally, this means that the shape of the function is dependent on the specific physical twin. In practice, however, relatively simple facts can already lead to useful improvements (see Section 3.1.3). For example, if it is known *a priori* that a specific phenomenon is present in the system, we could define

$$R_1(M) = \begin{cases} r_1, & \text{if } M \text{ contains a compartment} \\ \text{extending a building block of that phenomenon,} \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

where r_1 is a hyperparameter that signifies the relative importance of

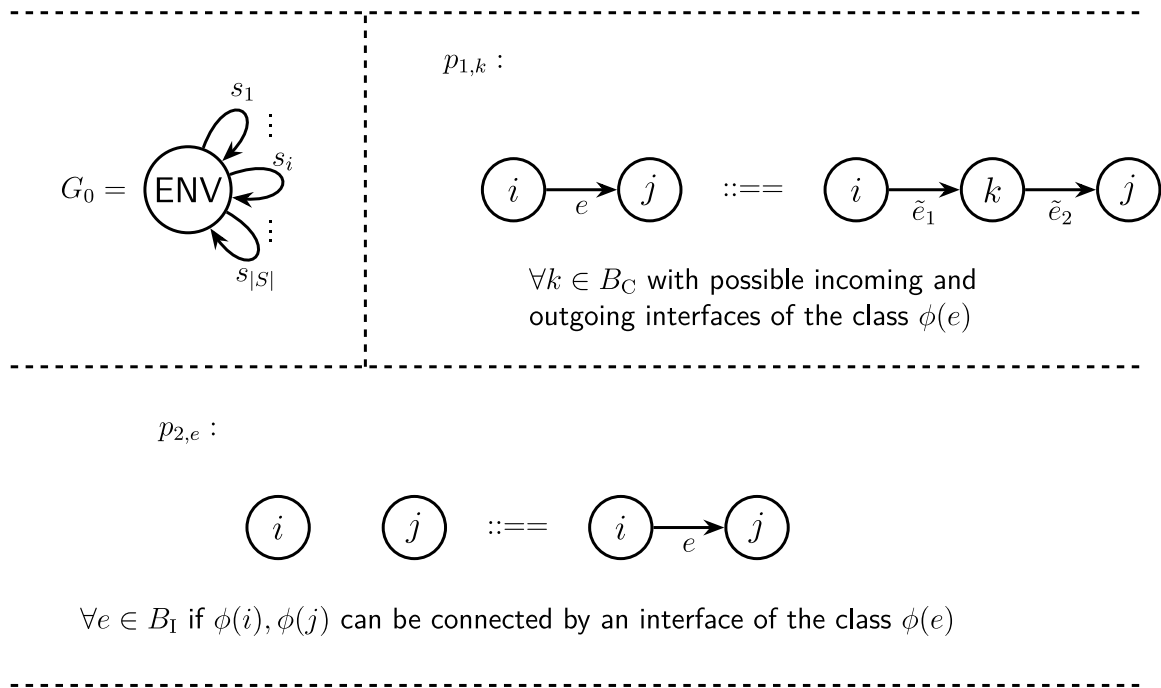


Fig. 5. An illustration of the basic graph grammar for mechanistic model generation. The initial graph G_0 is connected to itself by interfaces corresponding to the streams S of the physical system. The productions p_1 enable the insertion of appropriate compartments into existing interfaces, while the productions p_2 enable existing compartments to be connected by appropriate interfaces.

this fact in comparison to the accuracy reward $R_2 \in [0, 1]$. Similarly, penalties can be assigned for unwanted topological characteristics, e.g., too many compartments, cycles, *et cetera*.

Calculating the accuracy-based reward R_2 is much more computationally expensive than the knowledge-shaped reward R_1 , because an optimization problem with the embedded simulation model is solved, which generally takes the form of a MINLP. To speed up the overall model generation process, we calculate the knowledge-based reward first and restrict the calculation of the accuracy-based reward to models that pass a threshold with respect to R_1 . Topologies that do not pass the threshold automatically receive an $R_2 = 0$. Thereby, we heuristically focus the computational effort on the most promising regions of the search space based on prior expert knowledge.

2.3. Framework integration

The methodology introduced in the previous sections constructs a search space for model topologies and defines formalizations to explore it in a structured manner. This could constitute a stand-alone framework for cases in which the model topology is the desired output, e.g., when a purely hydrodynamic model is desired. In that case, no addition of further constitutive relationships to the basic equation scaffold is necessary. However, the ambition of this two-part work is to establish a general framework for automatic modeling of any chemical system. Therefore, in Section 2.3.1, we integrate the newly developed compartmentalization module into the workflow of Part 1 and describe the hierarchical combination of the RL agents. Finally, in Section 2.3.2, the informational basis of the methodology is extended to integrate system knowledge with useful artifacts from topology and equation generation.

2.3.1. Mechanistic modeling with hierarchical reinforcement learning

In Part 1, Heyer et al., 2025 conceptualize a workflow consisting of seven modules, each representing a task in the systematic modeling process. As explained in Section 2.1 and Section 2.3.1, the compartmentalization and equation generation module jointly generate candidate models of the physical system. The first step is achieved by setting up a scaffold of balance and coupling equations via the model topology, while the second step involves determining appropriate additional constitutive relationships and parameters.

The decisions made in the compartmentalization and equation generation modules are fundamentally different, which is reflected in the states and actions of their MDPs. Whereas the states in the compartmentalization module are the topology graphs that are manipulated by graph productions, in the equation generator, the states are a list of the model's mathematical symbols, which are substituted, neglected, or marked for parameter estimation. These differences motivate the distribution of decision-making over different hierarchical levels. As has been successfully demonstrated for automated process design (Khan and Lapkin, 2022), hierarchical reinforcement learning lends itself very well to optimizing over these heterogeneous and hierarchically organized search spaces. Reinforcement learning has also been widely applied in process control for sequential decision-making in dynamic, uncertain environments (Devarakonda et al., 2025; Hoskins and Himmelblau, 1992). This proven effectiveness in related chemical engineering domains motivates its use here for system identification, where similar challenges of noisy data, nonlinear dynamics, and the need for adaptive strategies must be addressed. Therefore, we choose hierarchical reinforcement learning to execute our MDPs. Supervised methods of machine learning are generally not applicable due to the lack of training data for modeling novel unit operations. The extrapolation from models of existing unit operations with machine learning could be an interesting future research direction.

For each iteration of the topology generation (upper-level), the equation generation (lower-level) is executed multiple times. After a finite number of lower-level iterations, the accuracy of the best-found

fully specified model constitutes the accuracy-driven part of the upper-level's reward scheme presented in Section 2.2.3. The connection of the two MDPs is visualized in Fig. 6.

The reinforcement learning on both MDPs is implemented as a tabular Q-learning approach, where the expected reward function Q_π is estimated by successively aggregating the reward of the explored state-action trajectories; see Part 1 (Heyer et al., 2025) for further details. Other reinforcement learning architectures, e.g., based on (graph) neural networks (Fan et al., 2019), could offer better scalability and data efficiency.

For the topology generation, the problem environment is initialized with the ENV compartment and the physical streams, as seen in Section 2.2.2. The RL agent then chooses from the available graph productions either randomly or informed by the estimated reward function. The probability of choosing a random action in each step is determined by the hyperparameter ϵ_U . Consequently, the probability of following the best estimated policy in a step is $(1 - \epsilon_U)$.

For the lower-level agent, only a fully defined state, i.e., where each physical quantity was the subject of a decision by the agent, can be evaluated, as only this leads to a complete mathematical model. This naturally leads to the division of the reinforcement learning into separate episodes, where each episode starts with the initial state, the completely undefined series of symbols, and ends with a fully specified model. Thus, each intermediate state-action pair is not able to generate an individual reward, but episodes are evaluated as a whole.

In contrast, for the upper-level agent, each state, i.e., the model topology generated along a trajectory, can be fully evaluated. Therefore, an upper-level episode has no natural ending and could, theoretically, continue indefinitely. In the following, we will assume that accurate models are not excessively complicated and principally lie in the vicinity of $n_{\max}$ steps of the starting topology. With this, the episodes of the topology generation take on more of the nature of batches that are common in the training of RL agents (Lange et al., 2012). Each episode can generate more than one specified model and thus more than one accuracy-based reward. Within one episode, we will average the individual reward of each state with the rewards of the states that follow it. This means that the reward assigned to a particular action in a state also depends on the quality of the states that it can lead to further along a trajectory, which enables the agent to navigate beyond points of local optimality.

The integrated, hierarchical reinforcement learning procedure on the MDPs defined in Section 2.2 was implemented in the Python framework set up in Part 1. The candidate models are automatically constructed in the Pyomo framework (Hart et al., 2011) using the differential-algebraic extension (Nicholson et al., 2018) when necessary.

2.3.2. Ontological knowledge integration

At various points in the workflow, system knowledge is integrated into automatic procedures to reduce the dependency on expert intervention, e.g., with specialized graph grammar or reward shaping. The RL agents can access this digitized knowledge via an ontology. Ontologies are structured frameworks for organizing information and knowledge within a specific domain, like process systems engineering (Bates, 2017). They consist of a set of concepts and relationships that define the entities in that domain and how they interact. By formalizing knowledge in this way, ontologies allow for effective data integration, sharing, and analysis across different systems and applications, which is used here to enable the flexible generation of models for different physical systems and operating conditions.

Our starting point is the process ontology OntoCAPE (Morbach et al., 2007), which is an extensive collection of various classes and relationships for chemical processes, unit operations, and physicochemical phenomena. We assume that for a given chemical process, such a process ontology exists and is instantiated with sufficient objects representing the conditions and phenomena that should be captured by the model. In our schema, this ontology represents knowledge about the physical

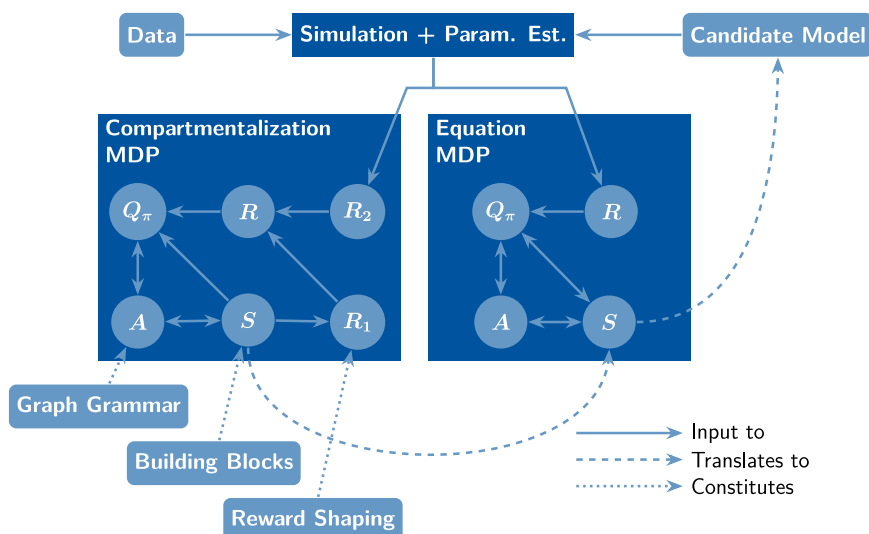


Fig. 6. Connection of the two MDPs for compartmentalization and equation generation. As introduced in Section 2.2, the constitutive units of the compartmentalization MDP are formalized as graph grammar, building blocks, and reward shaping. The model topology resulting from compartmentalization is used as the initial scaffold of equations in the equation generator and is therein fully specified to a candidate model. The candidate model is evaluated in simulation and parameter estimation and informs further manipulations of both the topology and the system of equations.

realm and constitutes one of two subdomains of our overarching knowledge graph. The second subdomain relates to the virtual realm. It provides all necessary classes for representing a compartmentalized model and artifacts of model generation that are interesting to store and examine across multiple model generation runs. Five types of artifacts are considered. The first two types concern knowledge already defined in Part 1 that is used directly for the equation generation, namely the libraries of possible constitutive equations and parameters to describe the system's phenomena. In this part, three more methods for knowledge inclusion and structured model exploration have been designed, i. e., building blocks, graph grammar, and reward shaping, which aid in the topology generation.

The modeling subdomain can be subdivided into three groups of classes that each fulfill a different role in the automated modeling process. The first group consists of all classes relating to mathematical objects like variables, parameters, operators, equations, or systems of equations. Single variables, parameters, operators, and numbers are abstracted as differential-algebraic-equation nodes ("DAENode") and constitute the subunits of the equations. Their arrangement is assumed to be in postfix notation and represented by different "eqPos" properties that each indicate the position of a specific symbol in a specific equation. The translation from postfix notation to valid Pyomo syntax is handled by a reverse shunting yard algorithm as described in Part 1. The second group represents common building blocks and phenomena that are standardized and stored across multiple model generation runs and different systems. Each "BuildingBlock" possesses one "MathModel" which represents a general instance of balance equations and boundary constraints that enable the translation from model topology to equation system. A "Phenomenon" object describes known or anticipated reactions, mass transfers, *et cetera* from the process subdomain and can also possess mathematical models. As opposed to the building blocks, phenomena can possess multiple variations of constitutive equations or parameters, that are explored by the equation generation module. The third group of classes are the actual constitutive subunits of individual topological models, i.e., compartments and interfaces, that each extend a "BuildingBlock" and inherit its predefined "MathModel". This way, specific model topologies can be stored and revisited over multiple model generation campaigns. Both the rules of graph grammar as well as reward shaping can be implemented as reasoning rules, such as SWRL rules. With that, valid manipulations and the rewards of stored topologies can be induced automatically and combined with knowledge from

the process subdomain. Fig. 7 shows a visualization of the ontology concept. An exemplary implementation was set up in the ontology software Protégé (Musen, 2015) but not yet integrated into real-time applications.

The benefit of using this ontology for digital twin applications is the decoupling of knowledge management from specific modeling campaigns. For example, it is beneficial to follow the ontology to generate a model for a novel unit operation for the first time and for successive recalibrations, as candidate models can be saved, set in relation to operating conditions (via the process subdomain), reevaluated, and automatically learned from by the RL agents. We outline such an example for the second case study. Additionally, the ontology offers a generic way to manage fundamental mechanistic model building blocks and graph grammar, see Fig. 5, which can be used for different purposes. In this way, new modeling tasks can benefit from digitized knowledge and expertise from previous campaigns.

3. Results and discussion

To test and evaluate the extended workflow, two case studies with different characteristics have been constructed. The first case study is a phase transfer catalysis reaction, for which a model has been adapted from literature, and data were generated through simulation. The purpose is to assess the efficiency of finding accurate models and to evaluate them against the known simulation model. The second case study is a complete automated modeling of a Taylor-Couette reactor, integrating both reaction yield and hydrodynamic measurements from experiments. Different operating points are examined to analyze the effect of dynamically complex system behavior on the model generation process, and transfer learning has been explored to recalibrate the model within digital twin applications.

3.1. Case study 1: phase transfer catalysis

Phase transfer catalysis (PTC) processes are characterized by multiple reactions occurring in different phases with a catalyst diffusing across the shared interfaces. For this *in silico* case study, we adapt the model developed by Helmdach et al. (2017) for the epoxidation of limonene. An Ishii-Venturello catalyst is dissolved in an aqueous solution of hydrogen peroxide together with a phase transfer agent. In this model, it is assumed that the catalyst and phase transfer agent are

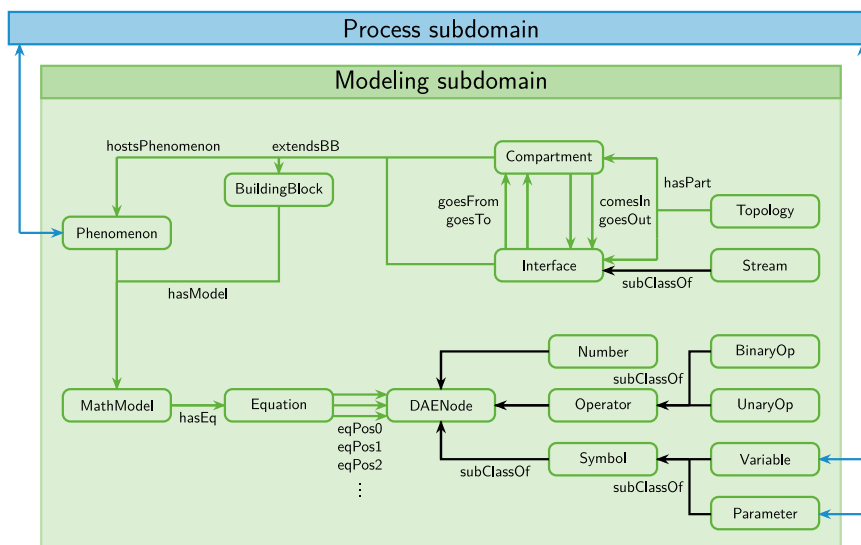


Fig. 7. A schematic illustration of the ontology concept for storing model topologies, general building blocks and phenomena, and their mathematical descriptions. Green arrows signify object-property relationships, while black arrows signify inheritance. The blue arrows indicate which elements of the “Modeling subdomain” generally relate to objects within the “Process subdomain” that describe the physical system and overarching engineering knowledge.

bonded permanently and constitute a single species with an active state (denoted by a bar) and a passive state. The active pair diffuses into the organic phase and catalyzes the epoxidation ($\text{Limonene} + \overline{\text{Catalyst}} \rightarrow \text{Epoxide} + \text{Catalyst}$), while the inactive catalyst is transferred back into the aqueous solution to be oxidized and reactivated by the hydrogen peroxide ($\text{Catalyst} + \text{H}_2\text{O}_2 \rightarrow \overline{\text{Catalyst}} + \text{H}_2\text{O}$). Possible side reactions from epoxide to diepoxide and diol were found to be negligible at the considered reaction conditions (Helmdach et al., 2017), and are excluded from this case study.

The simulation model used to generate reference data points can be found in Section 1 of the supplementary information. The outlet

concentrations of the epoxide product and the remaining limonene are simulated for a range of reactor temperatures. Six noiseless points from each curve are provided to the workflow for the following automatic identification of a suitable model.

3.1.1. Workflow initialization

The graph grammar, as shown in Fig. 5, as well as the building blocks and additional graph production in Fig. 8, are provided to the workflow. In particular, the topology generation agent can choose to include mixing blocks in the model topology. In this case, the mixing block represents an intense mixing of the phases so that the resulting streams exhibit quasi-homogeneous behavior. Although not envisioned in the

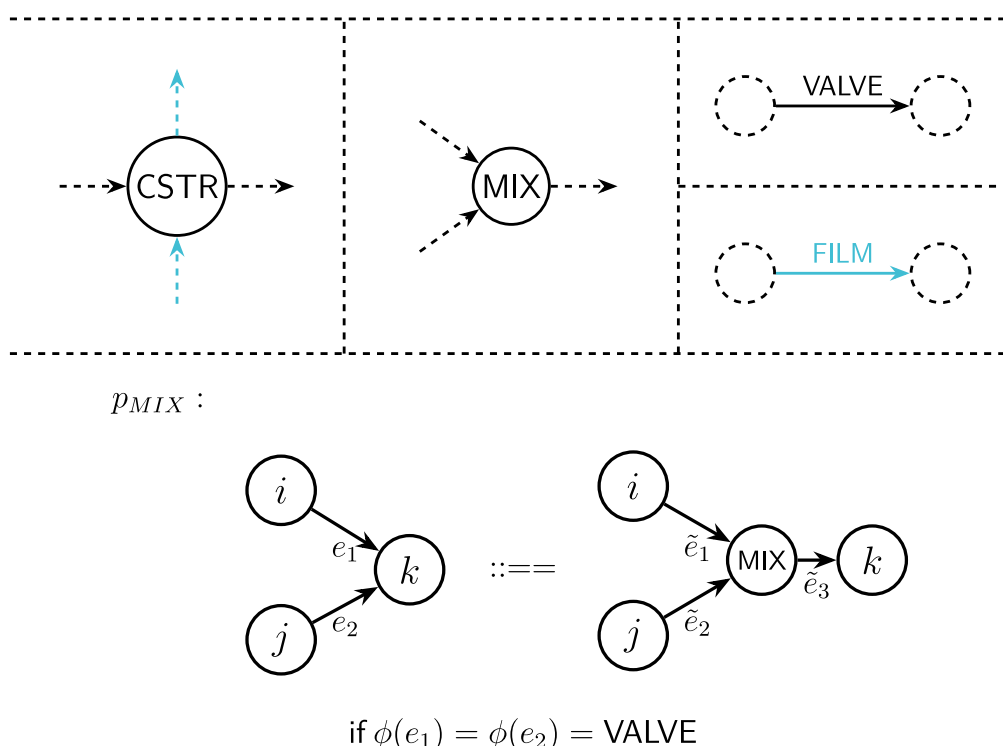


Fig. 8. Provided building blocks for case study 1 (phase transfer catalysis) and an additional graph production for handling mixing units.

original simulation model by Helmdach et al. (2017), this could represent an alternative description of the system.

As explained in Section 2.1, each topological object is translated into standardized balance and coupling equations by the information contained in its building block. The equations corresponding to the building blocks used in this case study are:

CSTR

$$b_m = F_{in} - F_{out} + J_{int} + J_{ext}, \quad (10)$$

$$J_{int} = V R, \quad (11)$$

MIX

$$b_m = F_{in,1} + F_{in,2} - F_{out}, \quad (14)$$

$$b_v = \dot{V}_{in,1} + \dot{V}_{in,2} - \dot{V}_{out}, \quad (15)$$

$$J_{ext} = \begin{cases} A J_{diffusion}, & \text{if connected to FILM interface} \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

$$b_v = \dot{V}_{in} - \dot{V}_{out}, \quad (13)$$

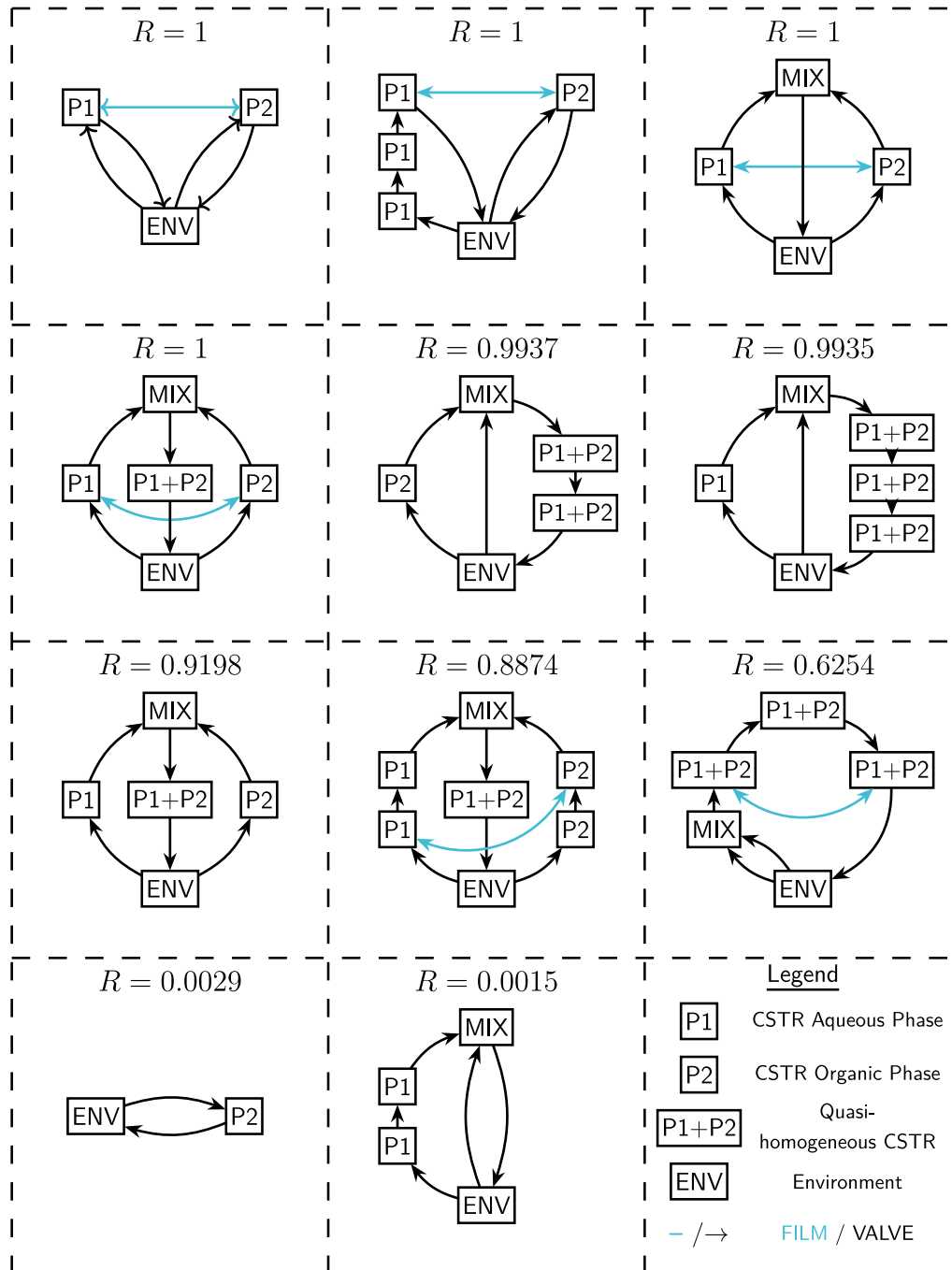


Fig. 9. A selection of automatically identified model topologies for the PTC reaction in case study 1 and their achieved rewards.

VALVE

$$s_m = F_{\text{out},c_s} - F_{\text{in},c_t}, \quad (16)$$

$$s_v = \dot{V}_{\text{out},c_s} - \dot{V}_{\text{in},c_t}, \quad (17)$$

FILM

$$s_{\text{diffusion},1} = J_{\text{diffusion},c_s} - J_{\text{diffusion}}, \quad (18)$$

$$s_{\text{diffusion},2} = -J_{\text{diffusion},c_t} - J_{\text{diffusion}}. \quad (19)$$

After the topology is compiled into a scaffold of balance and coupling equations, the lower-level equation generator sequentially decides for each symbol to be calculated as a state, estimated as a parameter, or substituted by a constitutive equation or a known parameter. The constitutive equations and parameter values from the simulation model have been adapted accordingly and are also provided to the workflow (see SI Section 1). If the equation module decides to utilize all provided information, the parameter estimation has no remaining degrees of freedom and reduces to the simulation of the outlet concentrations. Otherwise, the symbols that were automatically selected as parameters are optimized with respect to the error function. Initially, the distributed reward is based solely on the model error, as shown in Eq. (8), with respect to the twelve selected data points from the simulation.

3.1.2. Automatic generation of multi-phase models

In the following, the PTC case study is used to test the methods established in Section 2. The three objectives are to assess: (1) the quality of the automatically generated models, (2) the speed and reliability of finding good models, and (3) the influence of hyperparameters and heuristics, especially with respect to the use of formalized knowledge.

Fig. 9 shows a selection of models encountered by the RL agents during an exemplary run. The model in the top left corner represents the simplest way the information from the simulation can be represented with the given building blocks, and in that regard, it perfectly replicates the true model. Correctly, the workflow awards this model the best possible reward of 1. Other models achieve the same reward. In the case of the first row, the resulting topologies quantitatively express the same model but needlessly include excessive compartments, which is not explicitly prohibited. As these are different descriptions of the same model, here a human expert could make the final decision on which model should be used. Alternatively, including too many compartments could be penalized by appropriate shaping of the reward function.

The first model of the second row achieves a reward of one but includes a compartment where a quasi-homogeneous mixture of the phases hosts the reactions. This does not represent the true model adequately and would motivate us to include knowledge-based reward shaping that discriminates against arrangements like this. Naturally, other models with lower rewards have been explored during the search space exploration. Many of them represent a wrong, quasi-homogeneous approximation of the system and, correspondingly, achieve an overall lower reward. Too simplistic models, like in the bottom row, fail to replicate the experimental data entirely and are reprimanded accordingly.

Fig. 10 shows the tally of model rewards encountered over a completely random exploration of the search space. We see that the most simplistic models are visited most often, followed by the true model and slight deviations from it. The region of medium-reward models is severely underrepresented, demonstrating the sparse nature of the search space. This indicates that searching based on accuracy alone will be difficult, as learning to exploit this reward-sparse field is particularly challenging for stochastic optimization procedures like RL. This

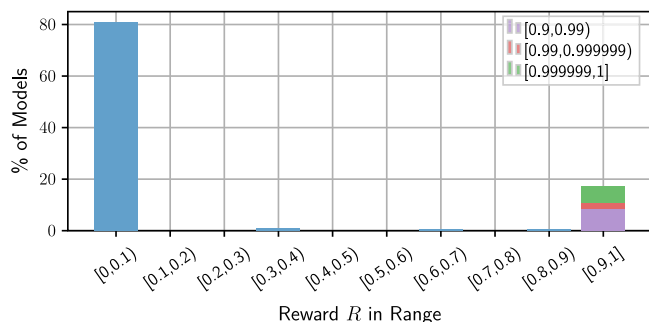


Fig. 10. Percentage of models in different reward brackets during a random search for the PTC reaction in case study 1. Total number of models: 448.

motivates us to examine possible improvements to the search procedures, like biasing the search space by reward shaping.

3.1.3. Analysis of workflow performance

To quantify the speed and reliability of finding accurate models, different scenarios of hyperparameters and heuristics were examined. For each scenario, the workflow was executed 50 times. For every run, the number of total episodes was capped at 20, and the maximum number of steps n_{max} within an episode was set to 10. The workflow was implemented in Python and executed single-threaded on an Intel Xeon 8468 Sapphire 2.1 GHz computing cluster. The models were automatically set up in Pyomo syntax by the implemented workflow and solved to global optimality by BARON (Sahinidis, 1996).

Scenario 1. Exploration versus Exploitation

Five different values for the probability of taking random steps ϵ_U were tested to quantify the impact of exploration versus exploitation on search efficiency. A value of $\epsilon_U = 1$ corresponds to taking completely random actions from each state. The lower the value of ϵ_U , the higher is the probability of choosing the best previously encountered action in that state based on the recorded state-action pairs in the Q-table. No formalized knowledge is provided for reward shaping, i.e., the reward is calculated from model accuracy alone. Figure 11(a) shows the frequency of finding the truth model over time for the different values of ϵ_U . We can see that with a higher degree of exploitation relative to random exploration, the RL finds the true model more slowly. It is apparent that any effort allocated to learning is reducing the efficiency of the search. From Fig. 10, we expect that behavior, as the search space is particularly sparse, with only a few models promising good rewards among an otherwise flat space of equally bad models. Due to a lack of gradients, such a search space is particularly hard to explore strategically, which we see reflected here. In contrast, the more reward-dense search space of the equation generation can be explored efficiently with a gradually decreasing $\epsilon = 1 \rightarrow 0$ as shown in Part 1.

Scenario 2. Reward Shaping

The inefficiencies of scenario 1 motivate us to include system knowledge in the search procedures. For example, a reward function that punishes topologies that do not include a diffusive mass transfer stream can be used:

$$R_{1,\text{mt-required}}(M) = \begin{cases} -1, & \text{if } M \text{ does not contain a FILM interface,} \\ 0, & \text{otherwise.} \end{cases} \quad (20)$$

Based on the knowledge about phase transfer catalysis, this is a reasonable requirement that all sensible models should fulfill. Under these conditions, the search is performed much faster, especially when some learning-based exploitation of the search space is enabled, see Fig. 11(b). This indicates that agents can learn to exploit knowledge-based biases in the search space, focusing their efforts on regions

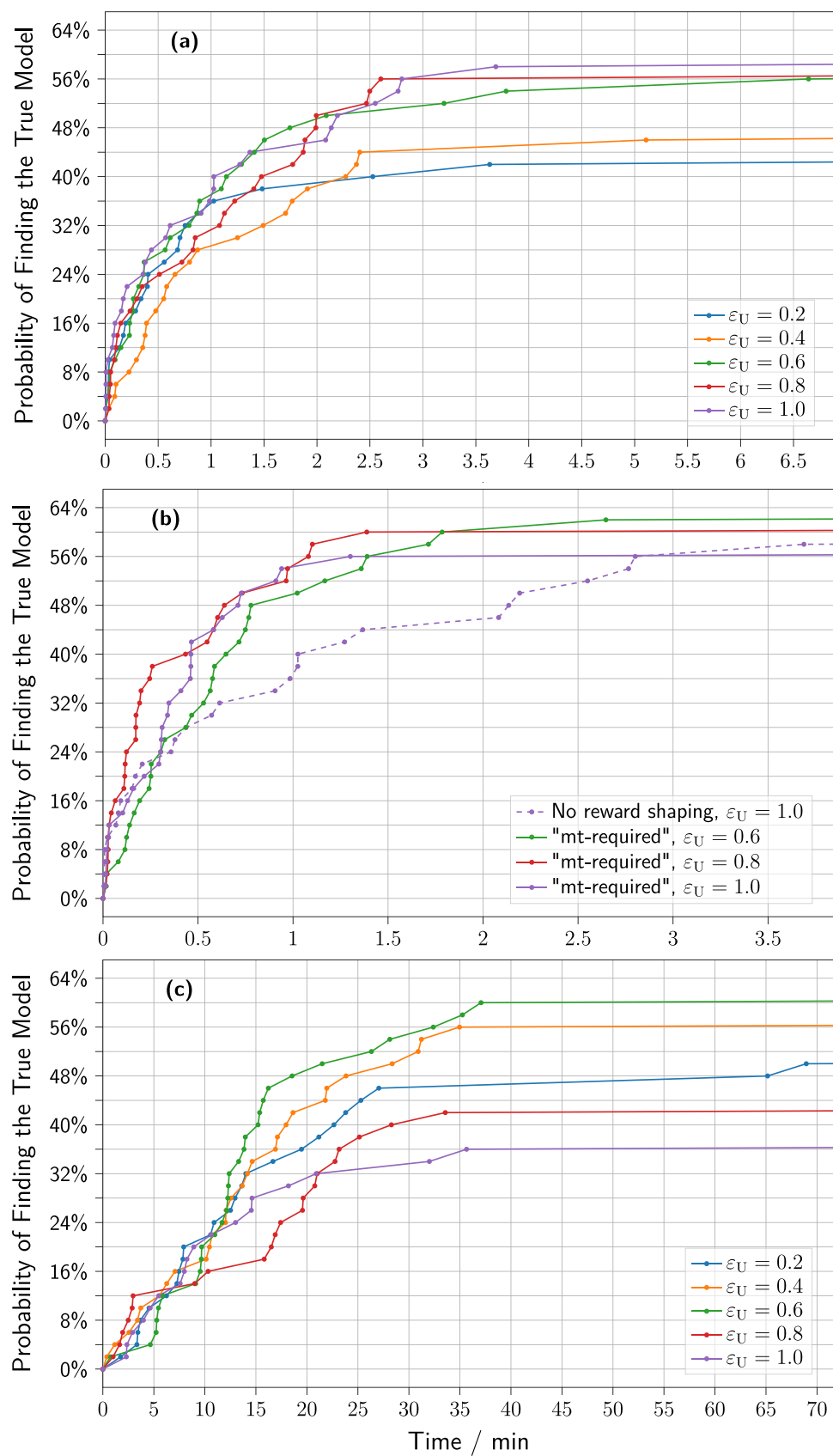


Fig. 11. Performance of the workflow under three scenarios, (a) Scenario 1: Exploration versus Exploitation, (b) Scenario 2: Reward Shaping, and (c) Scenario 3: Integrated Topology and Equation Generation.

where expert knowledge yields reasonable expectations of mechanistically more realistic models. The improvement in the case of random exploration with reward shaping stems from the heuristic that only topologies passing a threshold with respect to R_1 , in this case 0, are subjected to computationally expensive parameter estimation; see Section 2.2.3. This effect is the primary driver of finding the true model more quickly in this example. Another approach to improve computational time with expert knowledge is the tailoring of the graph grammar to the modelled system. This can reduce the size of the search space significantly but is less applicable to the characterization of novel unit operations, as a high degree of unit-specific modeling knowledge is required *a priori*. An illustrative example can be found in the supplementary information (Section 2).

Scenario 3. Integrated Topology and Equation Generation

For this scenario, the library of possible constitutive equations on the lower-level equation generation has been expanded to include alternative reaction kinetics. Therefore, the algorithm not only finds an adequate topological representation of the system but also explores the possible combinations of equations, akin to the case studies in Part 1. For the lower-level exploration, a maximum of ten episodes of the equation generator are executed, and $\epsilon_L = 0.4$. Fig. 11(c) shows the quantitative performance of the workflow under these circumstances. As expected, the workflow takes significantly longer to terminate, as up to 10 times more simulations are carried out for each step in the topology generation. Interestingly, we see a completely different effect of upper-level learning as compared to scenarios 1 and 2. A medium degree of learning leads to a significantly better performance than a random search. This can be explained partly because the search space for this scenario is slightly less reward-sparse. Another compounding effect is that for a higher degree of learning, good topologies are revisited more frequently, allowing the lower-level equation generator more opportunities in total to find the correct assembly of constitutive equations and parameters.

Overall, the introduced methods often find the true model within a timeframe of minutes, with the main difficulty being the sparsity of the search space. Sparsity stems from the specifics of this case study, as multiple phenomena must be included in just the right way such that an automatically generated model's predictions can come close to the true model from the base simulation. For applications of this type, the learning-driven part of the search space exploration should mainly be used to exploit knowledge-based biases and effects from the multiple levels of decision-making, and is, without these, not superior to random exploration.

The maximum reported probability of finding the true model of around 60 % is due to the cap on the maximum number of episodes, as we primarily seek to show feasibility and the general effect of exploration versus exploitation and knowledge biases. To improve overall performance, the reinforcement learning architecture should be enhanced with more efficient approaches, such as artificial neural networks and actor-critic structures (Fan et al., 2019). The implementation of the tabular Q-learning herein serves as proof-of-concept and further studies of the workflow's performance under real-life scenarios are necessary.

3.2. Case study 2: Taylor-Couette reactor

Taylor-Couette reactors (TCRs) are specialized chemical reactors that utilize the distinctive flow dynamics that occur between two coaxial cylinders, with the inner cylinder rotating. The achievable flow patterns range from Couette flow to more intricate structures like Taylor vortices and wavy vortex flows, based on the operating conditions of rotation rate and flow rate. Due to these adjustable flow regimes, TCRs have found application in innovative chemical and pharmaceutical processes (see for example Bouchon et al., 2023). For this case study, a TCR with a ribbed rotor was examined experimentally (see SI Section 2). Both

hydrodynamic data and reaction yield data will be used in the automatic generation of a model for digital twin applications.

3.2.1. Workflow initialization

Here, the experimental data gathered from the physical reactor is used to automatically generate candidate models. First, only residence time distribution (RTD) data is considered in the search for an accurate hydrodynamic model. Thus, the workflow is executed with a focus on the newly developed topology generation, and the equation generator is bypassed. For the compartments and interfaces, the standard building blocks for RTD modeling, as depicted in Fig. 12, are used. The tracer injection is modeled as a Dirac impulse with a concentration scaled to the variable volume of the first reaction space after the inlet, i.e., $c_{in}(t) = (m_{in}/V_1)\delta(t)$. The reactor volumes of CSTRs and PFRs, as well as split ratios at potential SPLIT compartments, are considered as the optimization variables with respect to the error function. Furthermore, the basic graph grammar in Fig. 5 was extended to allow for bypasses, as shown in Fig. 12, since these are commonly used to model non-ideal flow behavior. The initial graph includes only the environment compartment and an interface corresponding to the physical stream of the tracer. The sum of all building block volumes is constrained by the total physical volume of 17 mL, within a tolerance of ± 2 mL, to account for dead zones and measurement uncertainties. No reward shaping beyond the accuracy-driven reward is applied.

The workflow is executed single-threaded on an Intel Xeon 8468 Sapphire 2.1 GHz computing cluster. The maximum number of steps n_{max} within every episode was set to 20. As opposed to the first case study, here the resulting candidate models are solved without a guarantee of global optimality by DICOPT (Kocis and Grossmann, 1989). This is because of their larger size and the incorporation of mixed-integer variables for the PFR compartments. The setting used for DICOPT is "stop = 0", which forces the solver not to terminate until it finds the best integer-feasible model that can be reached by its internal procedures. Solving the models to local optimality is common practice in the evolutionary fitting of RTD models (see for example Laquerbe et al., 2001), and from examination, the local solutions are generally of sufficient quality for further processing.

3.2.2. Automatic generation of hydrodynamic models

For the residence time distribution resulting from a flow rate of 17

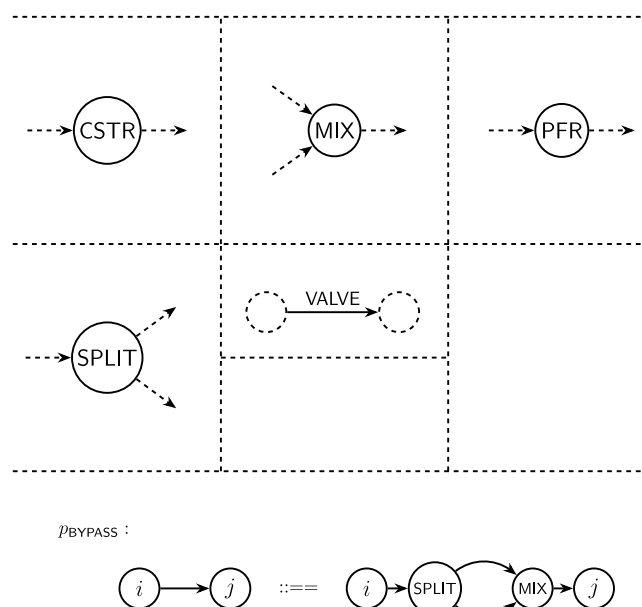


Fig. 12. Building blocks for predicting residence time distributions, and a graph production for the inclusion of bypasses.

mL min^{-1} and a rotation speed of 360 rpm, the model depicted in Fig. 13 was automatically identified. The workflow found it in episode 9 after evaluating 177 candidate models and approximately 20 min of runtime. The simulation and parameter fitting of the model itself takes 39 s. The model has an absolute root-mean-square deviation (RMSD) of $4.37 \times 10^{-5} \text{ g l}^{-1}$, and a relative RMSD of 5.4 %. Fig. 13 additionally shows the graphical data fit of the model. In this case study, the search procedures visit more diverse candidate models overall, as the search space is more reward-dense than in the first case study.

Similar to the multi-level reactor design methodology by Freund et al. (2019), we went from generalized modeling building blocks to reactor networks that can be interpreted with technical meaning. Thus, we can now compare the automatically generated systemic model from Fig. 13 with bespoke models from literature. First, we reflect on the established approaches for modeling TCRs. The first common approach is an axial diffusion model that represents the macro-mixing over the reactor length with a single dispersion coefficient (see for example Enokida et al., 1989, and Part 1). Although such models often sufficiently capture the overall reactor behavior, multi-compartment models have been developed as a second, more detailed way to represent and analyze the zonal characteristics of TCRs (see for example Ohmura et al., 1997). Here, the vortices resulting from the rotation are defined as separate compartments, and the mass transport between them is modeled as convective and diffusive fluxes.

Within the class of multi-compartment models, we can roughly identify three different topological designs. The simplest one represents a purely empirical fit of a cascade of CSTRs with identical volumes (Färber et al., 2016). The number of CSTRs is determined analytically from an approximation of the RTD curve. The next, more complex, approach uses more information about the actual geometry of the rotor and either implicitly (Richter et al., 2008) or explicitly (Giordano et al., 2000) considers bypass streams. For the case of ribbed rotors, Richter et al., 2008 further subdivide each vortex into upcoming and downcoming zones. The most sophisticated compartment models for TCRs not only consider the subdivision of the axial flow into vortices but further

represent each vortex as a layered structure with different intra-vortex transport phenomena between the core and the outer layers (Desmet et al., 1996). With the increasing detail of these models, the requirements for system information and detailed tracer experiments grow.

Our automatically identified model largely reflects the structure of the first two multi-zonal approaches. Similar to the empirical fit, we observe that the dominant part of the model consists of a cascade of CSTRs with identical volumes. Note that nowhere in our procedures is it explicitly prescribed for the compartment volumes to be identical, and that this property has been determined by the RL agents themselves. Although our workflow does not yet provide detailed geometric information, as in the second multi-zonal approach, a bypass flow, albeit relatively small, has been identified. In our model, we see the inclusion of a relatively large PFR in series. Mathematically, this shifts the outlet concentration forward in time by the residence time of this idealized reactor unit. We can interpret this PFR in two non-exclusive ways: On the one hand, the volume represented by the PFR could represent actual tube-like parts of the reactor setup, i.e., the tubing between the tracer injection and the reactor inlet, the reactor outlet, and the measurement cell, or PFR-like regions within the reactor before or after the rotor engages the flow. Considering the geometry of the reactor setup, this contribution should be comparatively small. On the other hand, the PFR could act as an approximation of a cascade of CSTRs. In that function, this building block significantly reduces the size of the model, which is a welcome side effect, albeit one that introduces mixed-integer variables.

In conclusion, we see that on a comparable informational basis, our automatic methods produce models very similar to the bespoke and expert-driven modeling methods from literature. We have also developed the necessary formalizations in the form of building blocks and graph grammar so that more detailed models like the layered structures from the most sophisticated approaches could be reproduced automatically if more system knowledge were provided. With that, we could extend the reach of our methods to true compartmental models as a progression to the systemic models presented here.

3.2.3. Multi-phenomena data integration

Next, we allow for the inclusion of two sets of data over the different RL hierarchies: First, RTD data will be used to fit the model topology M to the hydrodynamics of the reactor. Then, reaction yield data will be included to specify the reaction and mass transfer phenomena within the compartments of the topology, resulting in fully specified models $\hat{M}$. The reaction data is taken from previous experiments of an esterification reaction in the same reactor under the same operating conditions, which are presented in Part 1.

The best topologies generated from the RTD data are automatically selected, and their building blocks are replaced by equivalent blocks that enable the inclusion of the esterification reaction. For example, in contrast to the RTD calculation, the CSTR building block is now assumed to be in steady-state, and an internal flux is provided that can be specified as a reaction. The resulting scaffold of balance equations is then passed to the equation generator as its initial state. The equation generator performs a search for the best-fitting constitutive equations, parameter values, and parameter estimation decisions, and completely specifies the model. The library of possible equations and values is the same as in Part 1. In particular, four possible reaction kinetics of different orders (see SI Section 2) are provided, and the temperature dependency can be estimated directly or derived by the Arrhenius equation.

Based on the best hydrodynamic topology identified in the previous step, see Fig. 13, the integrated workflow finds two candidate models that can accurately predict the reaction yield as well. They differ with respect to the chosen reaction order and are summarized in Table 1. Inclusion or exclusion of the Arrhenius relationship has no further effect. The RMSD of the identified candidate models is comparable to the error

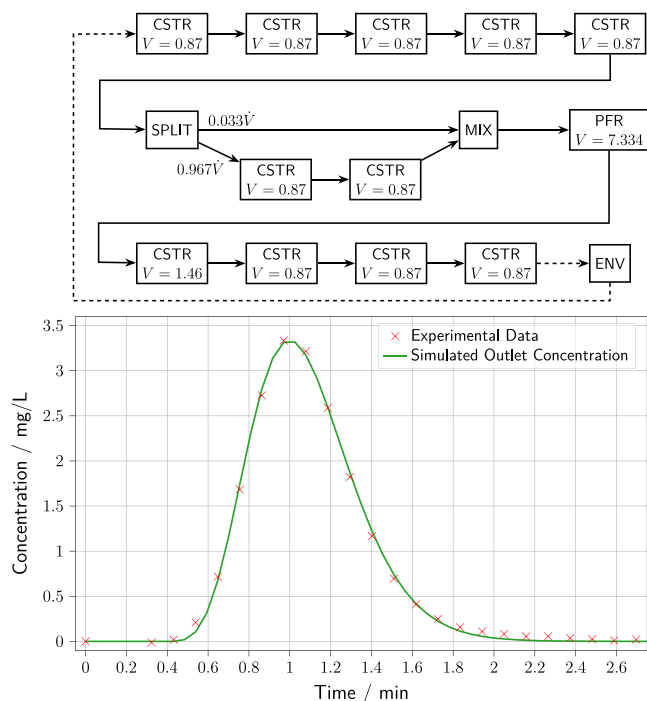


Fig. 13. Best found topology and data fit for a Taylor Couette reactor (TCR) with a total flowrate $\dot{V} = 17 \text{ mL min}^{-1}$ and a rotating speed $n = 360 \text{ rpm}$. V in mL.

Table 1

Evaluation of the two most accurate models generated for the Taylor-Couette reactor (TCR).

	Model $\hat{M}_1$	Model $\hat{M}_2$
Topology	Fig. 13	Fig. 13
Reaction order	{1,0,0}	{1,1,0}
Kinetic constant k	0.0056 s ⁻¹	0.0678 L mol ⁻¹ s ⁻¹
RMSD	1.1 × 10 ⁻³ mol L ⁻¹	1.1 × 10 ⁻³ mol L ⁻¹

of the diffusion-based models in Part 1. To discriminate between $\hat{M}_1$ and $\hat{M}_2$, and to therefore determine the correct order of the reaction, further experiments would be necessary. Nevertheless, we have demonstrated that the hierarchically integrated topology and equation generation workflow can produce well-fitting models with reasonable predictions of hydrodynamic and chemical behavior.

3.2.4. Model recalibration through transfer learning

Next, we change the operating conditions of the TCR and test the workflow's capability to recalibrate the model. When comparing the cases of a rotating and a stationary inner cylinder, the experimentally obtained tracer distributions exhibit significant qualitative differences, as shown in Figure S4. This is supported by literature, as the rotational speed has a direct impact on the observed flow regime and can be analyzed by the dimensionless Taylor number. For 0 rpm, we compute $Ta = 0$, and no toroidal vortices should be observed. Therefore, we expect fundamental differences in the automatically generated model topologies as well.

Indeed, the workflow produces structurally different models when fed with the tracer data from stationary operation. The volume flow has been kept at $\dot{V} = 17$ mL min⁻¹. After approximately 13 min the workflow identified the model depicted in Fig. 14. It matches the experimental data well with an absolute RMSD of 3.62×10^{-5} g L⁻¹ and a

relative RMSD of 3.98 %. As opposed to the model for the rotating case, we see a much higher impact of bypass flows, resulting in the particular shape of the tracer distribution's tails. It appears to be much easier for the flow to break past the mixing zones along the now stationary rotor, indicating that these zones now have a much weaker influence on hydrodynamics as compared to the vortex case. Apparently, in this case, the reactor exhibits a more CSTR-like behavior, with most of the volume being represented by a single CSTR. This high degree of mixing likely results from the relatively high flow rate, as we see a further qualitative shift in the cases of 0 rpm and lower flow rates.

To the authors' best knowledge, no compartment model for ribbed TCRs with a stationary rotor has been published. This is to be expected as the main draw of TCRs lies in the variable flow regimes induced by the inner rotation. However, it is still very relevant to research the characteristics of the reactor without rotation, e.g., to better understand startup and maintenance phases. By saving the generated models in the ontology, future modeling campaigns can benefit from the generated insights. Connecting the generated insights with established expert knowledge, such as the different operating conditions of TCRs and the flow patterns they induce, can aid experts in analyzing and inferring phenomenological changes.

We further investigate the performance of the workflow for the purpose of keeping a model up to date in the event of unexpectedly changing operating conditions. We illustrate this using a hypothetical scenario: consider a production run with the TCR at a flow rate of 17 mL min⁻¹ and a rotation speed of 360 rpm. The reactor is set up, and the reaction initially runs under the desired conditions. The initial model describing the TCR was generated using the automated methods presented in this work. After two-thirds of the production run, an unexpected fault in the power supply of the rotor occurs, and the rotational speed falls to 0 rpm. To further accurately predict the reactor's behavior, the model must be recalibrated on a structural level. Simply reusing the model topology of the rotating case is not feasible, as it would result in a high relative RMSD of 14 %.

Fig. 15 shows the quantitative evaluation of two different strategies for automatically recalibrating the model from the rotating to the stationary case. For the blue line, the model generation workflow is run normally without any special initialization. In the case of the orange line, the model is initialized with the memory, i.e., the Q-table, of the previous model generation run for 360 rpm. To compare these strategies, the workflow was executed 50 times each, and the time it takes to identify a recalibrated model with an accuracy-based reward of $R_2 \geq 0.9$ is recorded.

We can clearly see that a workflow that is initialized with the experience of the previous run performs better than an uninitialized run. Even though the workflow is executed for two entirely different system behaviors, resulting in different model topologies as seen in Fig. 13 and Fig. 14, the workflow profits from utilizing previous experience. Essentially, the pre-trained run can follow established trajectories that lead towards sensible models and jump from models that have previously been semi-successful to models that now adequately describe the new behavior. Naturally, this requires the old and new models to be more related to each other than to the topology with which the search is initialized. In our case, this is likely, as the initial model only represents the environment with an incoming and outgoing stream. Care should be taken when applying this strategy to extremely different behaviors.

Our results indicate that the pre-trained RL agents can exhibit an effective degree of transfer learning, for example, when analyzing a unit operating under different operating conditions, different solvent systems, or over time. In conclusion, this scenario demonstrates that an integrated, automated model generation procedure that leverages knowledge previously generated by itself and/or human experts is a valuable and promising approach for generating and maintaining models for digital twin applications.

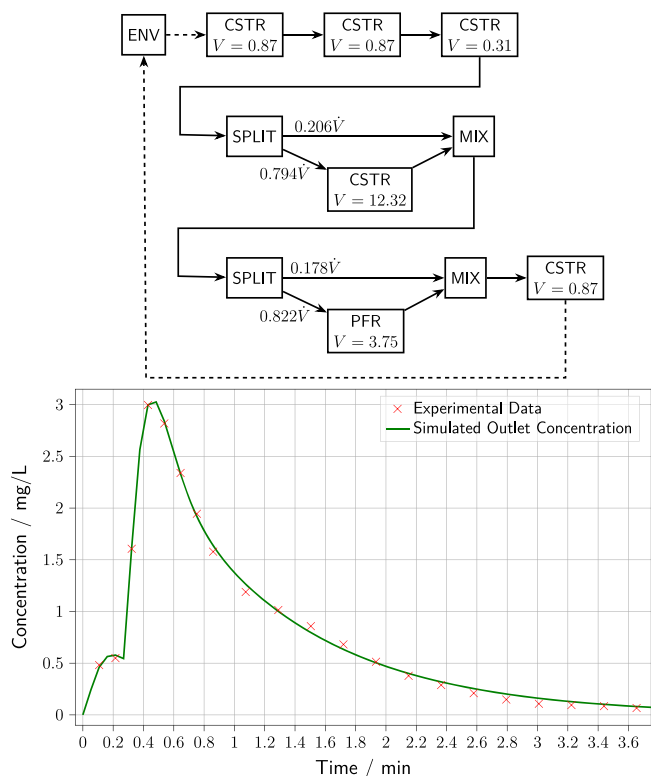


Fig. 14. Best found topology and data fit for a Taylor-Couette reactor (TCR) with a total flowrate $\dot{V} = 17$ mL min⁻¹ and a rotating speed $n = 0$ rpm. V in mL.

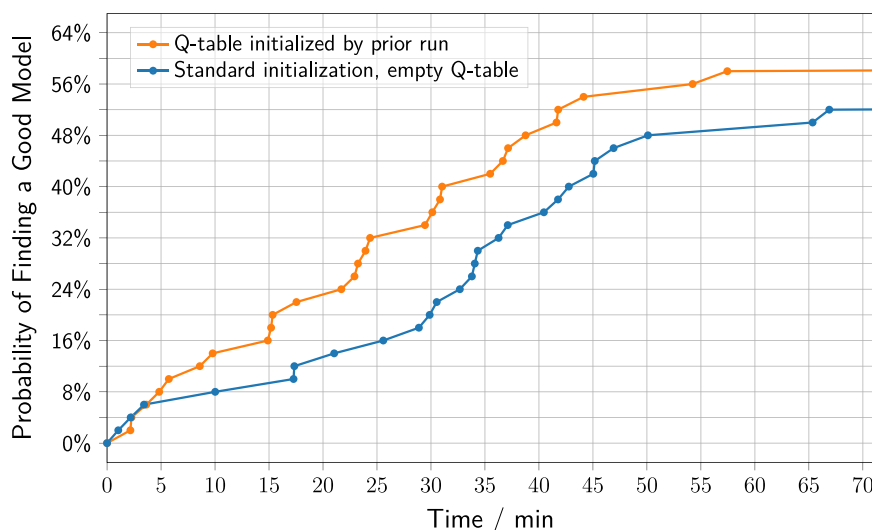


Fig. 15. Performance of the workflow initialized with an empty agent memory (blue) and with the Q-table of a prior run under different operating conditions (orange). The transfer learning is performed from the case of a rotating inner cylinder to a stationary one.

3.3. Current limitations and potential improvements

While the workflow offers promising capabilities for the automated generation of mechanistic models, several extensions could be made to improve its performance. The required computational effort remains high for most real-time applications, which necessitate dynamic model recalibration in the order of seconds rather than minutes. The biggest improvements can be expected from implementing a more sophisticated RL architecture that potentially reduces the required number of model simulations and parameter estimation calculations, which are the main drivers of computational time. Instead of reducing the frequency of numerical solutions, one could also reduce their computational demand through the use of surrogate models, in particular for larger or more complex systems (Bárkányi et al., 2021).

Another aspect for future development is the post-processing modules that, for example, assess the uncertainty of the automatically generated models. While Part 1 shows some robustness of the RL agents against aleatory uncertainty introduced by noisy data, and this contribution shows some management of epistemic uncertainty from generating and interpreting varying model topologies, the overall uncertainty quantification still needs to be automated and integrated with the rest of the workflow. To that goal, already established, open-source modules could be integrated (see for example Plank and Simpson, 2024). Similarly, data-driven modules could be integrated into the workflow to extend the herein generated mechanistic models to hybrid models (see for example Merkelbach et al., 2022).

Although the workflow produces dynamic, updatable mechanistic models suitable for use as the modeling core of a digital twin, it is not yet embedded in a full digital twin framework. Established architectures, such as (Onaji et al., 2022; Örs et al., 2020), define additional layers beyond the process model itself. These include data integration pipelines for continuous synchronization with plant sensors, visualization interfaces for operator interaction, and interoperability modules for linking with enterprise systems. Integrating our workflow into such a framework would require, for example, standardized communication protocols for model-sensor data exchange, automated triggers for recalibration based on live plant KPIs, and compatibility with industrial infrastructures for digital twin deployments. The present contribution focuses on the modeling component, whereas future work could aim to embed it into such multi-layer digital twin ecosystems so that it can be deployed in operational environments where automated model generation, real-time updating, and life-cycle tracking occur seamlessly.

4. Conclusions

Digital twins for complex chemical systems require accurate and flexible mechanistic models. The herein introduced methods complete a holistic workflow for the automated generation of such models, where the generation procedures of the model's topology and its equations are integrated over two hierarchical levels of reinforcement learning. The model topologies are abstracted as colored digraphs, and the rules for their iterative manipulation are formalized along concepts of graph grammar. Reward-shaping has been introduced to exploit existing system knowledge in the exploration of the search space of candidate models. The agents' base of knowledge has been consolidated in an ontology where common topological building blocks, their arrangement rules, and reward schemes are related to system and domain knowledge. With that, the role of human experts can be shifted from actively modeling each iteration of the mechanistic model to being curators of knowledge from which the workflow can automatically generate and recalibrate accurate models for chemical systems. This is especially relevant for the design and characterization of novel unit operations.

In the two case studies, the extended workflow has been shown to effectively generate accurate and meaningful mechanistic models. The automatically determined model topologies align well with established approaches from the literature, and the resulting mathematical models are consistent with observed or simulated data. Experimental hydrodynamic residence time distributions are predicted within a relative RMSD of $\sim 5\%$. Furthermore, it was shown that different types of experimental data can be integrated over the workflow's hierarchical levels to achieve high accuracy regarding both hydrodynamic and reaction phenomena. Transfer learning was successfully applied by reusing pre-trained RL agents after an operational shift of the physical system, increasing the likelihood of identifying an accurate model in under 30 min by 16 percentage points. This exemplifies the aptness of this extended workflow for the regular physical-to-virtual interactions that are essential to digital twins. For real-time applications with response times of seconds, further improvements are needed to reduce computational time, such as utilizing more data-efficient RL methods.

With the adaptation of building blocks as the principal components of the models, practically all relevant phenomena of chemical engineering can be captured and arranged in their contextual networks. Given adequate sets of building blocks, the current methods could generate single- and multi-compartment models for a large variety of unit operations. Further studies could explore alternative representations of the model graphs, for example, via graph neural networks, such

that sub-patterns in the model topologies can be recognized and learned from (Stops et al., 2023). By formalizing the manipulation of the model topologies along the paradigm of graph grammar, in the future, adequate grammars could be collected or even induced automatically (Fürst et al., 2020) for different kinds of systems. The results of the first case study suggest that further enhancements to the reinforcement learning should be explored, especially regarding the reward-sparse objectives that arise from the combinatorics of some applications.

Together, Parts 1 and 2 of this work present a holistic and flexible workflow for the automated generation of mechanistic models for digital twins. With more advanced sensing and measurement technologies, the workflow could become a powerful tool for characterizing innovative chemical unit operations. In future work, it would be highly interesting to integrate the workflow into a physically automated system with real-time data updates and model-based control.

Funding

This work is in part supported by EPSRC funded project “EP/W031019/1 Bio-derived and Bio-inspired Advanced Materials for Sustainable Industries (VALUED)” and was enabled by the Innovation Centre in Digital Molecular Technologies (iDMT) at the University of Cambridge. JFL was partly funded by the ERASMUS+ program for the duration of his stay at the University of Cambridge.

CRediT authorship contribution statement

Jan-Frederic Laub: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Jiyizhe Zhang:** Writing – review & editing, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mathis Heyer:** Writing – review & editing, Software, Methodology, Conceptualization. **Alexei A. Lapkin:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank AUTICHEM Ltd. for providing the Taylor-Couette flow reactor. JFL thanks Professor Alexei A. Lapkin, Professor Alexander Mitsos, and Wolfson College for enabling this research opportunity. The authors thank Raphael Seidenberg for fruitful discussions on reinforcement learning.

Supplementary materials

Supplementary material contains the simulation model adapted from Helmdach et al., 2017 that was used in the first case study; an illustrative example of tailoring graph grammar to the first case study; the experimental setup and results of the residence time distributions of the Taylor-Couette reactor; details on the dynamic modeling of the Taylor-Couette reactor and its connection to the RL environment; a nomenclature of all mathematical symbols and abbreviations.

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.compchemeng.2025.109384](https://doi.org/10.1016/j.compchemeng.2025.109384).

Data availability

Data will be made available on request.

References

- Babai, L., 2016. Graph isomorphism in quasipolynomial time. In: STOC '16: Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing, pp. 684–697. <https://doi.org/10.1145/2897518.2897542>.
- Bárkányi, Á., Chován, T., Németh, S., Abonyi, J., 2021. Modelling for digital twins—Potential role of surrogate models. *Processes* 9, 476. <https://doi.org/10.3390/pr9030476>.
- Batres, R., 2017. Ontologies in Process Systems engineering. *Chem. Ing. Tech.* 89, 1421–1431. <https://doi.org/10.1002/cite.201700037>.
- Bloyet, N., Marteau, P.-F., Frénod, E., 2020. Scott: a method for representing graphs as rooted trees for graph canonization. In: Cherifi, H., Gaito, S., Mendes, J., Moro, E., Rocha, L. (Eds.), *Complex Networks and Their Applications VIII, Studies in Computational Intelligence*. Springer, Cham, pp. 578–590. https://doi.org/10.1007/978-3-030-36687-2_48.
- Bouchon, F., Bergel, A., Filali, A., Bouchez, T., Fayolle, Y., 2023. First electrochemical Couette-Taylor reactor for studying the influence of transport phenomena on electrochemical kinetics. *Chem. Eng. Sci.* 281, 119103. <https://doi.org/10.1016/j.ces.2023.119103>.
- Darvari, V.-A., Hailes, S., Musolesi, M., 2021. Goal-directed graph construction using reinforcement learning. *Proc. R. Soc. A* 477. <https://doi.org/10.1098/rspa.2021.0168>.
- De Carvalho Servia, M.A., Sandoval, I.O., Hii, K.K.(Mimi), Hellgardt, K., Zhang, D., Antonio Del Rio Chanona, E., 2024. The automated discovery of kinetic rate models – methodological frameworks. *Digit. Discov.* 3, 954–968. <https://doi.org/10.1039/d3dd00021h>.
- Demirel, S.E., Li, J., Hasan, M.M.F., 2019. A General framework for process synthesis, integration, and intensification. *Ind. Eng. Chem. Res.* 58. <https://doi.org/10.1021/acs.iecr.8b05961>.
- Desmet, G., Verelst, H., Baron, G.V., 1996. Local and Global Dispersion Effects in Couette-Taylor Flow - I. Description and Modeling of the Dispersion Effects. *Chem. Eng. Sci.* 51. [https://doi.org/10.1016/0009-2509\(95\)00381-9](https://doi.org/10.1016/0009-2509(95)00381-9).
- Devarakonda, V.S., Sun, W., Tang, X., Tian, Y., 2025. Recent advances in reinforcement learning for chemical process control. *Processes* 13, 1791. <https://doi.org/10.3390/pr13061791>.
- Ehrig, H., Habel, A., Kreowski, H.-J., 1992. Introduction to graph grammars with applications to semantic networks. *Comput. Math. Appl.* 23. [https://doi.org/10.1016/0898-1221\(92\)90124-Z](https://doi.org/10.1016/0898-1221(92)90124-Z).
- Enokida, Y., Nakata, K., Suzuki, A., 1989. Axial turbulent diffusion in fluid between rotating coaxial cylinders. *AIChE J* 35, 1211–1214. <https://doi.org/10.1002/aic.690350720>.
- Fan, Z., Su, R., Zhang, W., Yu, Y., 2019. Hybrid Actor-Critic Reinforcement Learning in Parameterized Action Space. *arXiv*. <https://doi.org/10.48550/ARXIV.1903.01344>.
- Färber, T., Riechert, O., Zeiner, T., Sadowski, G., Behr, A., Vorholt, A.J., 2016. Homogeneously catalyzed hydroamination in a Taylor–Couette reactor using a thermomorphic multicomponent solvent system. *Chem. Eng. Res. Des.* 112. <https://doi.org/10.1016/j.cherd.2016.06.022>.
- Freund, H., Maußner, J., Kaiser, M., Xie, M., 2019. Process intensification by model-based design of tailor-made reactors. *Curr. Opin. Chem. Eng.* 26, 46–57. <https://doi.org/10.1016/j.coche.2019.08.003>.
- Freund, H., Sundmacher, K., 2008. Towards a methodology for the systematic analysis and design of efficient chemical processes. *Chem. Eng. Process.* 47, 2051–2060. <https://doi.org/10.1016/j.cep.2008.07.011>.
- Fürst, L., Mernik, M., Mahnić, V., 2020. Graph Grammar Induction. *Adv. Comput.*
- Gargalo, C.L., Malanca, A.A., Aouichaoui, A.R.N., Huusom, J.K., Gernaey, K.V., 2024. Navigating industry 4.0 and 5.0: the role of hybrid modelling in (bio)chemical engineering's digital transition. *Front. Chem. Eng.* 6. <https://doi.org/10.3389/fceng.2024.1494244>.
- Gilles, E.D., Holl, P., Marquardt, W., 1986. Dynamische simulation komplexer chemischer Prozesse. *Chem. Ing. Tech.* 58, 268–278. <https://doi.org/10.1002/cite.330580403>.
- Giordano, R.L.C., Giordano, R.C., Prazeres, D.M.F., Cooney, C.L., 2000. Analysis of a Taylor-Poiseuille vortex flow reactor - II: reactor modeling and performance assessment using glucose-fructose isomerization as test reaction. *Chem. Eng. Sci.* 55. [https://doi.org/10.1016/S0009-2509\(00\)00052-X](https://doi.org/10.1016/S0009-2509(00)00052-X).
- Grieves, M.W., 2023. Digital twins: past, present, and future. In: Crespi, N., Drobot, A.T., Minerva, R. (Eds.), *The Digital Twin*. Springer, Cham, pp. 97–121. https://doi.org/10.1007/978-3-031-21343-4_4.
- Haag, J., Gentric, C., Lemaitre, C., Leclerc, J.-P., 2018. Modelling of chemical reactors: from systemic approach to compartmental Modelling. *Int. J. Chem. Reactor Eng.* 16. <https://doi.org/10.1515/ijcre-2017-0172>.
- Hart, W.E., Watson, J.-P., Woodruff, D.L., 2011. Pyomo: modeling and solving mathematical programs in Python. *Math. Program. Comput.* 3. <https://doi.org/10.1007/s12532-011-0026-8>.
- Helmdach, D., Yaseneva, P., Heer, P.K., Schweidtmann, A.M., Lapkin, A.A., 2017. A multiobjective optimization including results of life cycle assessment in developing bio-renewables-based processes. *ChemSusChem* 10. <https://doi.org/10.1002/cssc.201700927>.
- Heyer, M., Zhang, J., Sugisawa, N., Laub, J.-F., Lapkin, A.A., 2025. Automated generation of mechanistic models for chemical process digital twins using reinforcement learning part I: conceptual framework and equation generation. *Comput. Chem. Eng.* 202, 109281. <https://doi.org/10.1016/j.compchemeng.2025.109281>.
- Hoskins, J.C., Himmelblau, D.M., 1992. Process control via artificial neural networks and reinforcement learning. *Comput. Chem. Eng.* 16, 241–251. [https://doi.org/10.1016/0098-1354\(92\)80045-B](https://doi.org/10.1016/0098-1354(92)80045-B).

- Jourdan, N., Neveux, T., Potier, O., Kanniche, M., Wicks, J., Nopens, I., Rehman, U., Le Moulec, Y., 2019. Compartmental modelling in chemical engineering: a critical review. *Chem. Eng. Sci.* 210. <https://doi.org/10.1016/j.ces.2019.115196>.
- Junttila, T., Kaski, P., 2007. Engineering an efficient canonical labeling tool for large and sparse graphs. In: *Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments*. <https://doi.org/10.1137/1.9781611972870.13>.
- Khan, A.A., Lapkin, A.A., 2020. Searching for optimal process routes: a reinforcement learning approach. *Comput. Chem. Eng.* 141, 107027. <https://doi.org/10.1016/j.compchemeng.2020.107027>.
- Khan, A.A., Lapkin, A.A., 2022. Designing the process designer: hierarchical reinforcement learning for optimisation-based process design. *Chem. Eng. Process.* 180, 108885. <https://doi.org/10.1016/j.ces.2022.108885>.
- Kocis, G.R., Grossmann, I.E., 1989. Computational experience with dicopt solving MINLP problems in process systems engineering. *Comput. Chem. Eng.* 13, 307–315. [https://doi.org/10.1016/0098-1354\(89\)85008-2](https://doi.org/10.1016/0098-1354(89)85008-2).
- Lange, S., Gabel, T., Riedmiller, M., 2012. Batch Reinforcement Learning. In: Wiering, M., van Otterlo, M. (Eds.), *Reinforcement Learning. Adaptation, Learning, and Optimization*. Springer, Berlin, Heidelberg, p. 12. https://doi.org/10.1007/978-3-642-27645-3_2.
- Laquerbe, C., Laborde, J.C., Soares, S., Ricciardi, L., Floquet, P., Pibouleau, L., Domenech, S., 2001. Computer aided synthesis of RTD models to simulate the air flow distribution in ventilated rooms. *Chem. Eng. Sci.* 56. [https://doi.org/10.1016/S0009-2509\(01\)00292-5](https://doi.org/10.1016/S0009-2509(01)00292-5).
- Mah, R.S.H., 1983. Application of graph theory to process design and analysis. *Comput. Chem. Eng.* 7. [https://doi.org/10.1016/0098-1354\(83\)80014-3](https://doi.org/10.1016/0098-1354(83)80014-3).
- Mane, S., Dhote, R.R., Sinha, A., Thirumalaiswamy, R., 2024. Digital twin in the chemical industry: a review. *Digit. Twins Appl.* 1, 118–130. <https://doi.org/10.1049/dgt2.12019>.
- Mazuz, E., Shtar, G., Shapira, B., Rokach, L., 2023. Molecule generation using transformers and policy gradient reinforcement learning. *Sci. Rep.* 13, 8799. <https://doi.org/10.1038/s41598-023-35648-w>.
- Merkelbach, K., Schweidtmann, A.M., Müller, Y., Schwoebel, P., Mhamdi, A., Mitsos, A., Schuppert, A., Mrziglod, T., Schneckener, S., 2022. HybridML: open source platform for hybrid modeling. *Comput. Chem. Eng.* 160. <https://doi.org/10.1016/j.compchemeng.2022.107736>.
- Morbach, J., Yang, A., Marquardt, W., 2007. OntoCAPE—A large-scale ontology for chemical process engineering. *Eng. Appl. Artif. Intell.* 20. <https://doi.org/10.1016/j.engappai.2006.06.010>.
- Musen, M.A., 2015. The Protégé Project: a look back and a look forward. *AI Matters* 1, 4–12. <https://doi.org/10.1145/2757001.2757003>.
- Nicholson, B., Sirola, J.D., Watson, J.-P., Zavala, V.M., Biegler, L.T., 2018. pyomo.Dae: a modeling and automatic discretization framework for optimization with differential and algebraic equations. *Math. Program. Comput.* 10. <https://doi.org/10.1007/s12532-017-0127-0>.
- Ohmura, N., Kataoka, K., Shibata, Y., Makino, T., 1997. Effective mass diffusion over cell boundaries in a Taylor-Couette flow system. *Chem. Eng. Sci.* 52, 1757–1765. [https://doi.org/10.1016/S0009-2509\(97\)00012-2](https://doi.org/10.1016/S0009-2509(97)00012-2).
- Onaji, I., Tiwari, D., Soulatiantork, P., Song, B., Tiwari, A., 2022. Digital twin in manufacturing: conceptual framework and case studies. *Int. J. Comput. Integr. Manuf.* 35, 831–858. <https://doi.org/10.1080/0951192X.2022.2027014>.
- Örs, E., Schmidt, R., Mighani, M., Shalaby, M., 2020. A conceptual framework for AI-based operational digital twin in Chemical process engineering. In: *2020 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*. Cardiff, United Kingdom. IEEE, pp. 1–8. <https://doi.org/10.1109/ICE/ITMC49519.2020.9198575>.
- Pal, P.K., Hens, A., Behera, N., Lahiri, S.K., 2025. Digital twins: transforming the chemical process industry—A review. *Can. J. Chem. Eng.* 1. <https://doi.org/10.1002/cjce.25611>.
- Plank, M.J., Simpson, M.J., 2024. Structured methods for parameter inference and uncertainty quantification for mechanistic models in the life sciences. *R. Soc. Open Sci.* 11. <https://doi.org/10.1098/rsos.240733>.
- Richter, O., Hoffmann, H., Kraushaar-Czarnetzki, B., 2008. Effect of the rotor shape on the mixing characteristics of a continuous flow Taylor-vortex reactor. *Chem. Eng. Sci.* 63, 3504–3513. <https://doi.org/10.1016/j.ces.2008.04.003>.
- Sahinidis, N.V., 1996. BARON: a general purpose global optimization software package. *J. Glob. Optim.* 8, 201–205. <https://doi.org/10.1007/BF00138693>.
- Stankiewicz, A.I., Moulijn, J.A., 2000. Process intensification: transforming chemical engineering. *Chem. Eng. Prog.* 96, 22–34.
- Stops, L., Leenhouts, R., Gao, Q., Schweidtmann, A.M., 2023. Flowsheet synthesis through hierarchical reinforcement learning and graph neural networks. *AIChE J* 69. <https://doi.org/10.1002/aic.17938>.
- Tian, Y., Demirel, S.E., Hasan, M.M.F., Pistikopoulos, E.N., 2018. An overview of process systems engineering approaches for process intensification: state of the art. *Chem. Eng. Process.* 133, 160–210. <https://doi.org/10.1016/j.ces.2018.07.014>.